

Pavo Cloud

Technical Execution Report | AI for Business Track

Dhanvanth Lakshman, CFO · Viraj Gadeela, CTO · Harry Wang, CEO

1. What Pavo Cloud Is

Before a doctor can go ahead with a procedure, such as an MRI, a knee replacement or an expensive drug, the insurer has to agree to pay for it first. That permission step is called *prior authorization*. In principle, it is a check against the plan's coverage rules. In practice, it is a member of staff filling in a form, faxing it or retyping it into an insurer's portal, and then waiting around three to fourteen days, typically. The United States spends around \$35 billion a year on this, and most of it buys nothing: the great majority of requests are approved in the end, just late.

Pavo Cloud takes the person out of the start of that process. When a doctor places an order in the hospital's record system, our software assembles the request, signs it so the insurer can prove who sent it, and delivers it straight to the insurer's software. That side checks the signature, applies the insurer's own published coverage rules, and returns a decision naming the exact rule behind it. For a clear-cut request, no person is involved at any point.

What happens when a request is *not* clear-cut matters more. The software never guesses. If no coverage rule matches, if a signature fails, or if an appeal is not confident enough, the case stops and goes to a human reviewer with the whole file already assembled. No AI language model is allowed to decide an authorization, and the software cannot issue a final denial to a patient at all. Section 5 sets out that boundary in full. The company earns three ways: a fee per request, a monthly subscription per practice, and a yearly fee to insurers for the connection. Round 1 argued all of this was possible; this report is about the software that now does it.

2. What Changed Since Round 1

2.1 From an Argument to Working Software

All four planned phases are built and running, and the system now has a public website and a deployable build on top of them. Every figure in this report was measured against the committed code leading up to the round rather than estimated.

Table 1. What was built, in the order it was built.

Phase	What it does	Status
Phase 1	The core loop: a doctor's order becomes an approval with no person involved	Complete
Phase 2	Insure: shows a patient what a procedure will actually cost them	Complete
Phase 3	Identity and appeals: agents prove who they are, and denials are appealed automatically	Complete
Phase 4	Privacy proofs: show a patient qualifies without sending their medical details	Complete
Since	A public website, a deployable build, a second AI provider, and an interface pass for public release	Complete

2.2 What the System Does, Measured

Round 1 promised a decision in under five minutes. The software turned out to be a lot faster, because the slow part was never the thinking: it was the paperwork around it. Almost all of the time that remains is cryptography rather than deciding: each authorization signs two messages, the request and the response, and one signature costs about 42 milliseconds. Checking a signature costs a twentieth of a millisecond. A more modern signature scheme would cut the decision into single figures. We have not switched, because 89 milliseconds is already far below anything a person notices.

Table 2. Measured results.

What was measured	Result
A complete authorization decision, start to finish	88.6 milliseconds
One signature, of the two every decision needs	42.0 milliseconds
Checking a signature	0.05 milliseconds
Creating a privacy proof about a patient	401 milliseconds
The full six-step demonstration, front to back	11.0 seconds
Automated tests passing, out of 148	148

2.3 Two Corrections to Round 1

Two claims did not survive being built, however. On *speed*, Round 1 said “under five minutes”; the agents themselves take under a tenth of a second, so five minutes now covers the round trip once the hospital's record system and the insurer's network are included. On *profitability*, Round 1 implied the company breaks even on each sale in year one. Rebuilt from what the software costs to run, year one is a 31% gross margin, meaning 31 cents of every dollar is left after the direct cost of serving that customer. Early customers need a great deal of hand-holding. The 87% figure for year three held.

3. System Architecture 2.0

3.1 How One Request Moves Through the System

A doctor places an order in the hospital's electronic health record, which is the software that holds a patient's chart. That order starts everything below. No person touches any of the seven steps.

The two sides talk over a small protocol of our own, called ARIA. It is deliberately dull. Each message is an envelope carrying its type, the sender, a timestamp, and the medical details in FHIR, the standard electronic format for health records that insurers are already required to accept. What matters is that every envelope is signed with the sender's private key, over a digest of the contents. A digest is a short fingerprint computed from every byte of a message, so altering a single character breaks the signature. Each organisation registers its public key once, and the receiving side looks that key up and checks it before reading anything else. The keys are RSA-2048, a long-established scheme in which each party keeps a private key it never shares and publishes a public key anyone can verify against. The private half is never stored here: it is handed back once when the organisation is created, and only a fingerprint of it is kept. A message

that fails the check is refused outright and the refusal is written to the audit trail, so a rejection leaves as full a record as an approval does.

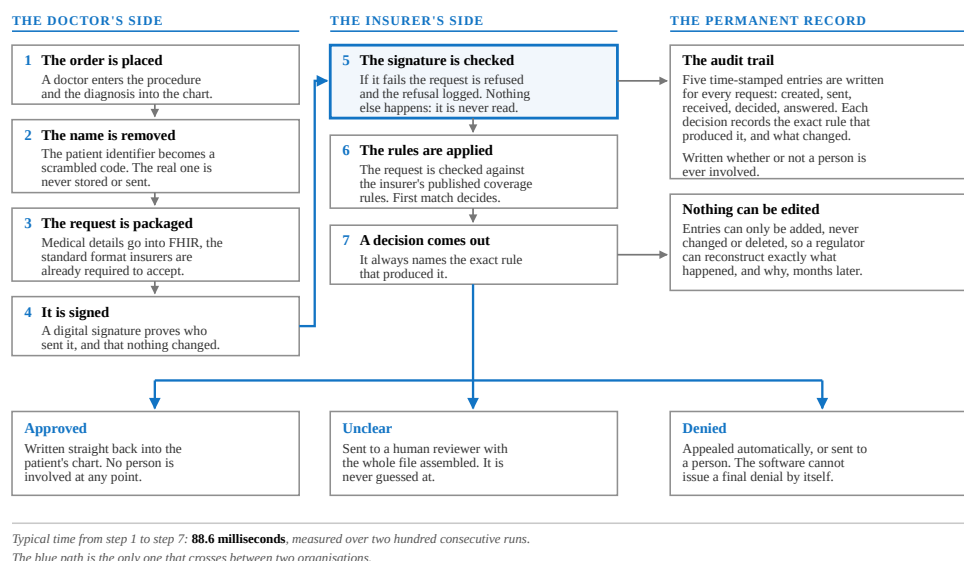


Figure 1. One authorization request, end to end. The heavier box at step 5 is a gate: if the signature does not check out, the insurer's software never reads the request at all.

3.2 Why Step 5 Matters

Two organisations letting software talk automatically only works if each can prove who it is. Step 5 is where that happens, and it happens *before* the request is opened: either the sender is proven genuine, or nothing happens. A judge can test this by changing any part of a request after it has been signed. It is refused outright.

3.3 Two Layers, and Which One Is Allowed to Decide

Round 1 described a fixed-rules layer and a learning layer. The built system draws that line harder than promised. **No AI language model ever decides an authorization.** Models are used only where being approximately right is acceptable, such as reading a photograph of an insurance card, and every result is scored before it is used.

Table 3. What each layer is allowed to do.

Layer	What it handles	The guarantee it gives
Fixed rules (decides)	Identity checks, coverage rules, price arithmetic, sorting denial reasons	The same input always produces the same answer, and every answer names the exact rule behind it. That is what makes a decision defensible to a regulator.
AI models (assist only)	Reading insurance cards and benefit documents, turning plain English into a medical code, drafting appeal letters	Every result carries a confidence score. Below the threshold it goes to a person. A model is never the last word on whether a patient gets care.

One consequence is worth stating plainly. If the AI provider went offline tomorrow, prior authorization would keep working, because there is no AI on that path. The same fact explains why processing one request costs a fraction of a cent (section 6.2).

3.4 When a Request Is Denied

A denial is never the end. The software sorts the insurer's stated reason, then acts on it.

Table 4. What happens after a denial.

Reason given	What the software does
Not covered by the plan	Stops. This is a contract question, not a medical one, so a person decides. No letter is drafted.
Missing information, not medically necessary, or any unrecognised reason	Searches the published medical literature, drafts an appeal citing the studies it finds, and scores its own confidence in it.
Confidence below 70%	The appeal is held for a person, with the diagnosis, the reason, the score and every study already attached.

3.5 What Happens to Patient Data

Patient identifiers are scrambled at the door, before anything is saved, and the scrambling is one-way: the original cannot be recovered from it. A test in the codebase checks that the real identifier never appears in storage.

The system goes further using a technique called a zero-knowledge proof. It lets the insurer confirm three things: that the patient is old enough, that the diagnosis is covered, and that the deductible condition is met. It does that *without receiving the age, the diagnosis or the amount*. One limitation is listed in Table 6.

4. The Build Log

4.1 What Was Committed, and When

The code lives in a repository called *hwangus922/AI2O-Pavo-Cloud*. Work was done in eleven batches: one per phase, a round of fixes, and six more since Round 2 opened. Each was reviewed before being merged. Together they come to 22,739 lines added across 260 file changes, 15,871 of them hand-written and the rest a dependency lockfile.

```
$ git log --first-parent --date=short --pretty="%h %ad %s" 0a24def

0a24def 2026-09-19 Document the setting that keeps the site private (#11)
bb02bc4 2026-09-19 Give the site a real mark and take every error surface off it (#12)
cd0891a 2026-09-19 Make a broken or stale deployment say so (#10)
ee9ed87 2026-09-18 Fix the broken request detail page, and make every outcome reachable (#9)
84c4ca5 2026-09-18 Turn the demo app into a full website (#8)
8b23760 2026-09-18 Make the demo deployable to a public URL, and support DeepSeek (#6)
a13f6cb 2026-09-12 Fix the three failures that break a deployed demo (#5)
abc1b5c 2026-09-09 Phase 4: demo flow, audit UI, and a real ZK proof (#4)
99bbb96 2026-09-08 Phase 3: autonomous appeals and cryptographic identity (#3)
5f26194 2026-09-07 Phase 2: Insure price transparency (#2)
e48ec54 2026-09-07 Phase 1: autonomous prior authorization core loop (#1)
d5ea3a7 2026-09-07 Initial commit

$ git log --shortstat 0a24def

18 commits · 260 file changes · +22,739 insertions · -1,215 deletions
```

Figure 2. The commit history. Each line is a batch of work that was reviewed and merged. The largest single batch is Phase 1 at 9,896 lines, though two thirds of that is a dependency lockfile rather than code anyone wrote; by hand-written lines the largest is Phase 4.

4.2 The Tests

An automated test is a small program that checks the main program still behaves correctly. There are 148 of them and they all pass. They matter more than the line count, because they are what turns a claim in this report into something a judge can verify with one command.

```
$ .venv/bin/python -m pytest

tests/test_appeals.py ..... [ 29%]
tests/test_flow.py ..... [ 41%]
tests/test_insure.py ..... [ 70%]
tests/test_persistence.py ..... [ 75%]
tests/test_rules.py ..... [ 85%]
tests/test_zk.py ..... [100%]

148 passed, 1 warning in 23.28s
```

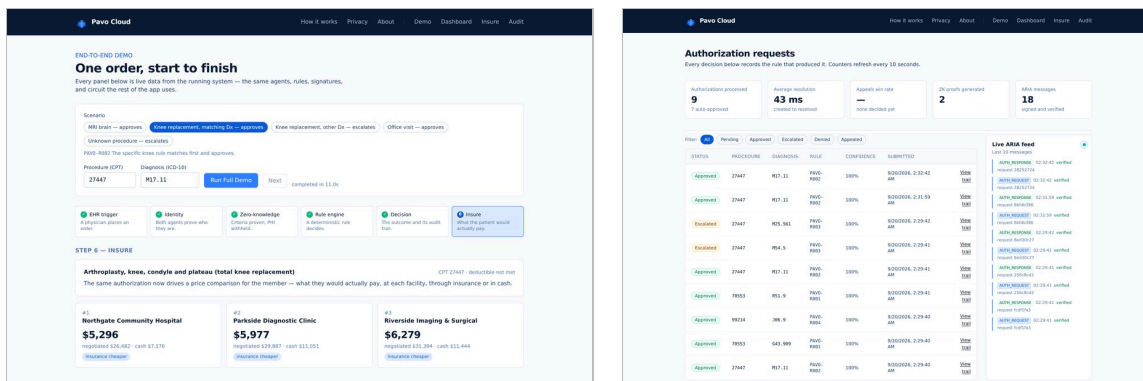
Figure 3. The full test suite; each dot is one passing test. Among the things they prove: every decision records the rule that produced it, an unmatched request goes to a human rather than being guessed at, a tampered message is refused, and a real patient identifier never reaches storage.

4.3 The System Running

The screens below are the live software, not mock-ups. Both were captured against the code in the log above.

Both are served by the same build that would go to a customer. The interface is a Next.js application, which is a standard framework for building websites in JavaScript, and it talks to the Python backend over exactly the interface an insurer's software would use. Nothing in either screen is a special demonstration mode.

One addition merged in the last week is visible here: a fallback to a second AI provider behind the assistive summaries, so an outage at one vendor cannot take those features down with it. It does not change how a decision is made. A third change is not visible: the operator-facing diagnostics added in #10 were taken back out of the public build before release, leaving a deploy-time guard that fails the build outright if it is configured without its backend address.



Figures 4 and 5. Left: the guided demonstration after a single click; all six steps (order, identity, privacy proof, rules, decision, price) run with no further input and finish in 11.0 seconds, and the last of them prices the same operation at all five facilities in the price file, showing the three cheapest; across all five the spread is \$2,475 for identical care. Right: the dashboard, with the request form that sits between the counters and the table left out here. Every row carries the rule that produced it, and the two amber rows are knee replacements whose diagnosis did not match the covered condition; both wait for a human reviewer with the file already assembled.

5. Risk Mitigation Protocol

5.1 The Governing Rule

One principle runs through every part of the system: **the safe outcome is the default, not the exception**. When the software is unsure it does not guess. It stops and hands the case to a person, with the file already prepared.

Table 5. What happens when something goes wrong.

If this happens	The software does this, automatically	A person is involved
No coverage rule matches the request	Marks it unclear and routes it to a human reviewer. There is no path by which an unmatched request is approved or denied.	Within 4 hours, insurer's clinical reviewer
A message has been tampered with	Refuses it before reading it, and records the refusal with the reason.	Immediate security alert
An appeal scores below 70% confidence	Holds it. Nothing is sent to the insurer. Reviewer notes are written automatically.	Within 1 working day, provider's appeals staff
The AI provider goes offline	Appeal drafting falls back to a placeholder scored zero, which is below the threshold, so every appeal escalates. Authorization itself is unaffected.	None required
The medical literature service is unreachable	Finishes the appeal with no citations and records the failure. Confidence drops, which pushes it to a person.	Follows the appeal route above
An appeal is rejected by the insurer	Marks it unclear. The software cannot issue a final denial to a patient.	Always, a licensed reviewer
Approval rates drift by demographic	Quarterly review across age, geography and diagnosis. Any rule causing a disparity is suspended to human review.	Quarterly, clinical and compliance

5.2 Why Human Review Is Not a Queue

Escalating a case is a routing decision, not a backlog. Every case handed to a person arrives complete: the medical details, the rule that fired, the signature check, the denial reason, the confidence score and any studies found. The bottleneck Round 1 identified was never medical judgement. It was the paperwork wrapped around it.

5.3 What We Have Not Solved

These are real limitations of a prototype. We would rather state them than be caught by them.

Table 6. Known gaps and what each needs.

Gap	What it needs
The privacy proof reveals which covered condition a patient has	A more advanced circuit design. Scoped for the next phase.
The proof's setup was done on one machine	A proper multi-party ceremony before any real use. Not safe for production as it stands.
Provider identity is trusted, not verified	A live lookup against the federal provider registry. One integration.

Facility prices are generated, not gathered	The voice agent that calls facilities, plus the price files insurers must now publish.
Five coverage rules, not a rule library	Clinical staff encoding each insurer's published criteria. This is the largest cost in our model and the real barrier to a competitor.
The learning layer is not built	Deliberate. Until it exists, anything without a definitive rule goes to a person, which is the correct default anyway.

6. Three-Year Financials

6.1 Where the Money Comes From

Pavo earns in three ways: a fee for each request, a monthly subscription per medical practice, and a yearly fee to insurers for the connection itself. The figures below are built up from customer counts and request volumes, not from a growth rate applied to a starting number. The full workbook accompanies this report.

Table 7. Revenue, built up from customers.

Source	2026	2027	2028
Medical practices served (average)	55	330	1,400
Insurers connected (average)	11	84	160
Requests processed	171,600	1,647,360	8,736,000
Fees per request, at \$3.00	\$514,800	\$4,942,080	\$26,208,000
Practice subscriptions, at \$400/month	\$264,000	\$1,584,000	\$6,720,000
Insurer connection fees, at \$20,000/year	\$220,000	\$1,680,000	\$3,200,000
Total revenue	\$998,800	\$8,206,080	\$36,128,000

The \$3.00 fee is 85–92% below the \$15–\$40 an insurer spends handling a request by hand today, which is what makes the switch easy to justify.

6.2 What It Costs to Run

Servers are not the expense. Processing one request costs about six hundredths of a cent, because the deciding path uses no AI models at all. The real cost is clinical staff translating each insurer's published rules into the fixed rules the software applies. That is the largest line in every year, and the reason margins improve with scale rather than with technology.

Table 8. Cost of serving customers, and what is left.

Cost	2026	2027	2028
Clinical staff writing coverage rules	\$189,000	\$455,000	\$1,080,000
Setting customers up	\$224,000	\$476,000	\$1,428,000
Customer support	\$92,000	\$190,000	\$784,000
Servers and storage	\$54,000	\$198,000	\$900,000
AI model usage (documents and appeals only)	\$11,000	\$62,000	\$280,000
Security and compliance certification	\$118,000	\$112,000	\$290,000
Gross profit	\$310,800	\$6,713,080	\$31,366,000
Gross margin	31%	82%	87%

6.3 Cost to Win a Customer, and What One Is Worth

Two standard measures. *Acquisition cost* is what it costs in sales and marketing to sign one customer. *Lifetime value* is the profit that customer produces before they leave, stated here over three years, which is deliberately conservative.

Table 9. Customer economics at 2028 rates.

Measure	Practice	Insurer	Blended
Cost to acquire one customer	\$6,000	\$22,000	\$6,699
Value over three years	\$58,248	\$51,057	\$57,533
Value per \$1 spent acquiring	\$9.70	\$2.30	\$8.60
Months to earn the cost back	3.5	15.2	4.0

At \$2.30 back per \$1 spent and fifteen months to recover it, insurer contracts do not pay for themselves as a product. We fund them anyway, because signing one insurer makes Pavo available to every practice that already submits to it. It is a distribution channel, not a profit centre.

6.4 Profit, and What a Request Actually Costs

After engineering, sales and admin, the company loses \$2.37 million in 2026 and \$3.21 million in 2027, then makes \$11.23 million in 2028. The deepest point is \$5.58 million of cumulative losses, in late 2027. The assumption most likely to be wrong is the number of practices, not the price. If all four of our main assumptions are wrong at once, 2028 revenue is \$13.8 million rather than \$36.1 million: a smaller company, but still a real one.

Two per-request costs appear in this report and they measure different things, so it is worth saying which is which. Dividing the cloud infrastructure line in Table 8 by the requests processed gives 31 cents in 2026, 12 cents in 2027 and 10 cents in 2028. That line covers compute, the database, object storage, egress and seven years of audit retention, so it charges every request for records that must be kept long after the request has been answered. The marginal cost of one more authorization is the compute alone: fixed rules over a single database read, no model call, a few milliseconds of processor time. That is the six hundredths of a cent quoted in section 6.2, and it is the figure that matters when volume doubles.

The model API line behaves differently again, falling from 6 cents per request to 3 across the three years. None of it is spent on the authorization path: those tokens read insurance documents and draft appeals. That is why Table 5 can say an AI outage leaves authorization unaffected, and why the cost of deciding does not move when model prices do.

7. How to Check Any of This

`git log --shortstat 0a24def` reproduces the build figures in section 4.1, and `.venv/bin/python -m pytest` reproduces the 148 passing tests in 4.2. Starting the backend and posting one request, with the command in the repository's README, returns the decision, the rule behind it and the confidence: the whole of section 3 in one response. The demonstration at `/demo` runs the entire path end to end in about eleven seconds, and `/audit` shows every message the two agents exchanged along the way.